

Dbms

Unit 5 introduction to transaction processing concepts

Q-1 ACID Properties of Transaction

ACID properties ensure that database transactions are processed reliably.

1. Atomicity (All or Nothing)

- A transaction is either fully completed or not executed at all.
- If any part fails, the whole transaction is rolled back.

Example:

Bank transfer of ₹1000

- Debit from A ✓
 - Credit to B ✗ (fails)
→ Entire transaction is cancelled → A gets money back
-

2. Consistency

- The database must remain in a valid state before and after the transaction.
- It follows all rules, constraints, and relationships.

Example:

If total balance in system = ₹10,000

After transfer, it must still be ₹10,000 (no money loss/creation)

3. Isolation

- Transactions run independently without interfering with each other.
- Intermediate states are not visible to other transactions.

Example:

Two users updating same account:

- One transaction should not see half-updated data of another
-



Edit with WPS Office

4. Durability

- Once a transaction is committed, changes are permanent.
- Even if system crashes, data is not lost.

Example:

After successful money transfer, even if power fails, data remains saved.

Q-2. Transaction Definition

Transaction

A **transaction** is a sequence of database operations (read/write) that performs a single logical unit of work.

Example:

Bank transfer =

1. Read balance
 2. Deduct amount
 3. Add amount to another account
 4. Save changes
-

Transaction State Diagram

States of a Transaction

Start → Active → Partially Committed → Committed
↓
Failed → Aborted → (Restart or End)

Explanation of Each State

1. Active

- Transaction is being executed (read/write operations).
 - Initial state.
-

2. Partially Committed

- All operations executed, but not yet saved permanently.
 - Waiting for final commit.
-

3. Committed



Edit with WPS Office

- Transaction successfully completed.
 - Changes are permanently stored.
-

4. Failed

- Transaction cannot proceed due to error (system crash, constraint failure).
-

5. Aborted

- Transaction is rolled back.
 - Database restored to previous consistent state.
-

6. Terminated

- Final state after commit or abort.
 - Transaction ends.
-

Simple Example of State Flow

Case: Successful Transaction

Start → Active → Partially Committed → Committed → Terminated

Case: Failed Transaction

Start → Active → Failed → Aborted → Terminated

Q-3 Characterizing Schedule Based on Serializability

A **schedule** is the sequence of execution of operations from multiple transactions. **Serializability** ensures that concurrent transactions produce the same result as some serial (one-by-one) execution.

Types of Serializability

1. Conflict Serializability

A schedule is conflict serializable if it can be converted into a serial schedule by swapping **non-conflicting operations**.



Edit with WPS Office

Conflicting operations:

- Read–Write
- Write–Read
- Write–Write (on same data item)

Example:

T1: R(A), W(A)

T2: R(A), W(A)

Schedule:

R1(A), R2(A), W1(A), W2(A)

This is **not conflict serializable** because order of writes matters.

👉 Use **precedence graph**:

- Node = transaction
- Edge = conflict

If graph has **no cycle** → **serializable**

2. View Serializability

Based on final result rather than conflicts.

Conditions:

- Same initial reads
- Same read-from relation
- Same final writes

More flexible than conflict serializability.

Conclusion

Serializability ensures **consistency and correctness** in concurrent execution.

Q-4 UML Class Diagram – Railway Reservation System

Main Classes

```
+-----+
| Passenger |
+-----+
| name      |
+-----+
```



Edit with WPS Office

age
gender

Ticket
ticket_id
status
seat_no

Train
train_no
name
route

Reservation
date
booking_status

Relationships

- Passenger → books → Ticket
- Ticket → belongs to → Train
- Reservation → manages booking

Explanation

- Passenger enters details
- Reservation system allocates ticket
- Train contains journey info
- Ticket confirms seat and status

Q-5 Role of Transaction Log in Recovery

A **transaction log** records all database operations.

Contents

- Transaction start
 - Data changes (before/after values)
 - Commit/abort status
-



Role in Recovery

1. Undo Operation

If transaction fails → use log to restore old values.

2. Redo Operation

If system crashes after commit → redo changes.

Example

T1 writes A=100 → 200

Log stores:

Before = 100, After = 200

- Crash before commit → Undo → A=100
 - Crash after commit → Redo → A=200
-

Conclusion

Transaction log ensures **data recovery and consistency** after failures.

Q- 6 Serializability in DBMS

Serializability ensures that concurrent transactions behave like serial execution.

Importance

- Maintains data consistency
 - Prevents anomalies (lost update, dirty read)
 - Ensures correctness
-

Types of Schedules

1. Serial Schedule

Transactions execute one after another.

2. Non-Serial Schedule

Transactions execute concurrently.



Edit with WPS Office

3. Serializable Schedule

Non-serial but gives same result as serial.

Types of Serializability

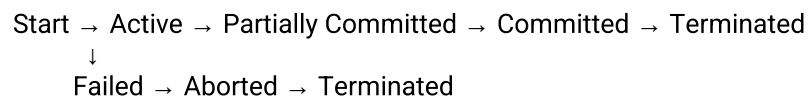
- Conflict serializability
 - View serializability
-

Conclusion

Serializability is key to **safe concurrent execution**.

Q-7 Transaction Lifecycle States

Diagram



States Explanation

1. Active

Transaction is executing operations.

2. Partially Committed

All operations done but not yet saved.

3. Committed

Changes are permanently saved.

4. Failed

Error occurs (system crash, constraint violation).

5. Aborted

Transaction rolled back to previous state.

6. Terminated



Edit with WPS Office

Transaction ends after commit/abort.



Edit with WPS Office